Báo Cáo Đề Tài:

Phân Loại Ảnh Quy Mô Lớn Sử Dụng Học Chuyển Tiếp (Transfer Learning) và Triển Khai Phân Tán Trên Apache Spark

Môn học: SE363 - Phát triển ứng dụng trên nền tảng dữ liệu lớn
Học viên / Sinh viên: Võ Minh Tiến
Mã số sinh viên (MSSV): 24521784

1. Giới Thiệu & Tổng Quan Kiến Trúc Hệ Thống

Trong các hệ sinh thái xử lý dữ liệu lớn hiện đại, việc thực hiện suy luận học sâu (Deep Learning Inference) trên quy mô hàng triệu bức ảnh đặt ra thách thức lớn về năng lực tính toán và khả năng mở rộng. Dù các mạng nơ-ron tích chập (CNN) mang lại độ chính xác vượt trội, việc xử lý thông lượng cao đòi hỏi kiến trúc phải vượt qua giới hạn phần cứng của một máy chủ đơn lẻ.

Để giải quyết bài toán này, đề tài xây dựng một hệ thống phân loại ảnh phân tán từ đầu đến cuối kết hợp giữa **Học chuyển tiếp (Transfer Learning)** và **Apache Spark**:

- **Học chuyển tiếp trên ảnh tự nhiên**: Khai thác biểu diễn đặc trưng thị giác sâu sắc từ mạng xương sống ResNet-18 đã được tiền huấn luyện trên ImageNet, kết hợp tinh chỉnh tầng phân loại tùy biến cho bài toán nhận diện 37 giống vật nuôi trên tập dữ liệu Oxford-IIIT Pet (7.349 ảnh màu).
- **Suy luận phân tán trên Apache Spark**: Tích hợp runtime PyTorch vào Apache Spark để thực thi dự đoán theo lô song song trên các phân vùng worker.
- **So sánh hai mô hình thực thi UDF**: Triển khai và đánh giá thực nghiệm hai kỹ thuật UDF cốt lõi trên Spark:
  1. **`pandas_udf`** dựa trên mô hình streaming scalar iterator với Apache Arrow.
  2. **`predict_batch_udf`** tận dụng API suy luận véc-tơ hóa theo lô nguyên bản của PySpark ML.
- **Phân tích thực nghiệm chuyên sâu**: Khảo sát toàn diện về độ chính xác, độ trễ, thông lượng (throughput), độ nhạy kích thước batch, hiệu suất song song và phân tích lỗi chi tiết.

2. Khảo Sát & Lựa Chọn Tập Dữ Liệu

2.1 Thông Tin Tập Dữ Liệu

- **Tên tập dữ liệu**: Oxford-IIIT Pet Dataset
- **Đường dẫn chính thức (URL)**: https://www.robots.ox.ac.uk/~vgg/data/pets/
- **Định dạng dữ liệu**: Ảnh màu 3 kênh RGB với kích thước tự nhiên đa dạng.

- **Tổng số lượng ảnh**: **7.349 ảnh**
- **Số lượng lớp**: **37 giống vật nuôi** (gồm 25 giống chó và 12 giống mèo)

2.2 Phân Chia Dữ Liệu & Tiền Xử Lý

Dữ liệu được phân chia theo cấu hình chuẩn từ tập chú thích gốc của Đại học Oxford:

Tập dữ liệu	Số lượng ảnh	Tỷ lệ	Mục đích sử dụng
Train (Huấn luyện)	2.944	40,0%	Tinh chỉnh tầng phân loại với kỹ thuật tăng cường dữ liệu
Validation (Kiểm định)	736	10,0%	Giám sát hội tụ và lựa chọn checkpoint tối ưu
Test (Kiểm tra)	3.669	50,0%	Triển khai suy luận và đánh giá phân tán trên Apache Spark
Tổng cộng	**7.349**	**100,0%**	Toàn bộ dữ liệu thực nghiệm

Quy trình Tiền xử lý & Tăng cường Dữ liệu (Pipeline):

- **Quy trình tập Huấn luyện (Train)**:

$$\text{Resize}(224 \times 224) \rightarrow \text{RandomHorizontalFlip}(p=0.5) \rightarrow \text{ToTensor}() \rightarrow \text{Normalize}(\mu, \sigma)$$

- **Quy trình tập Kiểm định & Kiểm tra (Val/Test)**:

$$\text{Resize}(224 \times 224) \rightarrow \text{ToTensor}() \rightarrow \text{Normalize}(\mu, \sigma)$$

- **Tham số chuẩn hóa theo ImageNet**:
  - Giá trị trung bình (Mean): $\mu = [0.485, 0.456, 0.406]$
  - Độ lệch chuẩn (Std): $\sigma = [0.229, 0.224, 0.225]$

3. Kiến Trúc Học Chuyển Tiếp & Quá Trình Huấn Luyện

3.1 Lựa Chọn Mạng Xương Sống (Backbone): ResNet-18

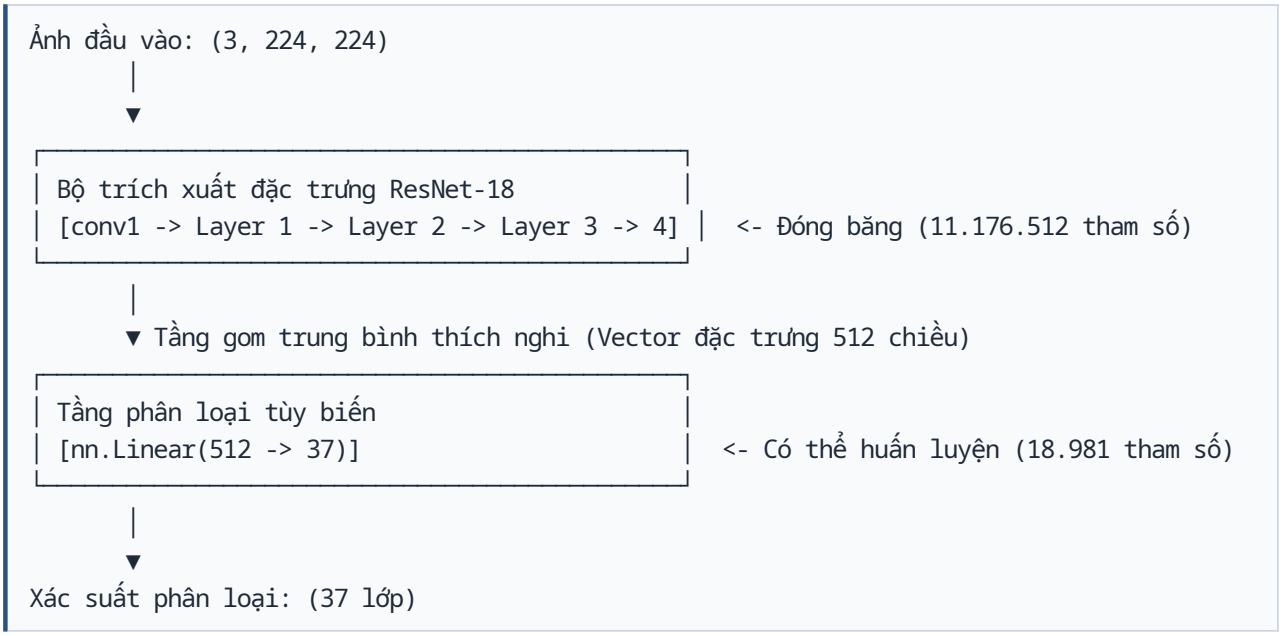
Mô hình sử dụng mạng **ResNet-18** (`torchvision.models.resnet18(weights=ResNet18_Weights.DEFAULT)`) đã được tiền huấn luyện trên ImageNet-1k (1,28 triệu ảnh, 1.000 lớp). ResNet-18 được chọn nhờ sự cân bằng xuất sắc giữa khả năng trích xuất đặc trưng thị giác phong phú và tốc độ tính toán cao, rất phù hợp khi suy luận trên tài nguyên CPU.

3.2 Tinh Chỉnh Tầng Phân Loại (Head Adaptation)

1. **Đóng băng tham số trích xuất đặc trưng**: Toàn bộ trọng số từ `conv1` đến `layer4` của ResNet-18 được đóng băng (`param.requires_grad = False`) nhằm giữ nguyên các bộ lọc đặc trưng đã học từ ImageNet và loại bỏ tính toán đạo hàm không cần thiết.

2. **Thay thế tầng kết nối đầy đủ (Fully Connected Layer)**: Thay thế tầng tuyến tính 1.000 lớp gốc bằng một tầng tuyến tính mới tương ứng với 37 giống vật nuôi:

$$\text{fc}_{\text{mới}} = \text{nn.Linear(in_features=512, out_features=37)}$$



3.3 Thiết Lập Huấn Luyện & Kết Quả Hội Tụ

- **Hàm mất mát (Loss Function)**: Cross-Entropy Loss đa lớp ($\mathcal{L}_{\text{CE}}$)
- **Thuật toán tối ưu (Optimizer)**: Adam ($\text{lr} = 10^{-3}$)
- **Kích thước lô (Batch Size)**: 64
- **Môi trường thực thi**: CPU đa nhân (20 luồng xử lý)

Tiến trình hội tụ qua các Epoch:

- **Epoch 1**: Train Loss: 2.1298, Train Acc: 51,56% | Val Loss: 1.1000, Val Acc: 76,49%
- **Epoch 2**: Train Loss: 0.8411, Train Acc: 83,73% | Val Loss: 0.6838, Val Acc: 84,51%
- **Epoch 3**: Train Loss: 0.5579, Train Acc: 88,38% | Val Loss: 0.5458, **Val Acc: 87,64%**

Mô hình đạt độ chính xác kiểm định **87,64%** chỉ sau 3 epoch huấn luyện trên tầng phân loại mới. Trọng số tốt nhất được lưu tại `checkpoints/best_model.pth`.

3.4 Xuất Định Dạng TorchScript Cho Môi Trường Phân Tán

Nhằm tối ưu hóa quá trình triển khai trên các tiến trình worker của Spark và loại bỏ các lỗi liên quan đến tuần tự hóa lớp Python (Python serialization / unpickling), mô hình được chuyển đổi sang định dạng TorchScript bằng phương pháp JIT Tracing (`checkpoints/model_scripted.pt`). Định dạng này cho phép môi trường C++ PyTorch thực thi trực tiếp trên worker mà không phụ thuộc vào mã nguồn định nghĩa lớp Python.

4. Triển Khai Mô Hình Phân Tán Trên Apache Spark

4.1 Cấu Hình Môi Trường Spark

- **Phiên bản Apache Spark**: 4.2.0
- **Master**: Cụm cục bộ (`local[*]`) khai thác toàn bộ luồng CPU.
- **Bộ nhớ Driver**: 4 GB
- **Tối ưu hóa dữ liệu**: Kích hoạt Apache Arrow (`spark.sql.execution.arrow.pyspark.enabled = true`).
- **Cấu trúc Spark DataFrame**: Tập kiểm tra (3.669 dòng) với cấu trúc schema (`image_path: String, label: Integer`), được phân vùng thành 4 phân vùng phân tán.

4.2 Phương Pháp 1: Triển Khai Bằng `pandas_udf`

PySpark `pandas_udf` tận dụng Apache Arrow để truyền tải các khối dữ liệu dạng cột (columnar batches) giữa tiến trình JVM và tiến trình Python worker. Nhằm tránh chi phí nạp lại mô hình cho từng dòng hoặc từng batch nhỏ, thiết kế áp dụng mẫu **Pandas Iterator UDF** (`Iterator[pd.Series] -> Iterator[pd.Series]`):

```
@pandas_udf(IntegerType())
def predict_pandas_udf(batch_iter: Iterator[pd.Series]) -> Iterator[pd.Series]:
    # 1. Khởi tạo và nạp mô hình DUY NHẤT một lần trên mỗi worker partition
    device = torch.device("cpu")
    model = torch.jit.load(model_path, map_location=device)
    model.eval()

    # 2. Xử lý dữ liệu dạng luồng (streaming) qua từng lô trong phân vùng
    with torch.no_grad():
        for path_series in batch_iter:
            batch_tensors = [preprocess(Image.open(p)) for p in path_series]
            logits = model(torch.stack(batch_tensors))
            predictions = logits.argmax(dim=1).cpu().numpy().astype(int)
            yield pd.Series(predictions)
```

Ưu điểm chính:

- Tránh nạp lại trọng số mô hình nhiều lần, giảm áp lực phân bổ bộ nhớ.
- Cơ chế iterator truyền trực tiếp dữ liệu dạng luồng, giữ mức tiêu thụ RAM ổn định.

4.3 Phương Pháp 2: Triển Khai Bằng `predict_batch_udf`

Được giới thiệu chính thức trong thư viện Spark ML (từ Spark 3.4+ / 4.x), hàm `predict_batch_udf` cung cấp tầng trừu tượng cấp cao dành riêng cho suy luận học sâu (Deep Learning Inference) theo lô:

```
def make_predict_fn():
    # Mô hình nạp 1 lần trên mỗi tiến trình worker
    model = torch.jit.load(model_path, map_location=torch.device("cpu"))
    model.eval()

    def predict(paths: np.ndarray) -> np.ndarray:
        # Nhận trực tiếp mảng NumPy theo lô và trả về mảng kết quả
        batch_tensors = [preprocess(Image.open(str(p))) for p in paths]
        with torch.no_grad():
            logits = model(torch.stack(batch_tensors))
            return logits.argmax(dim=1).cpu().numpy().astype(np.int32)

    return predict

predict_batch = predict_batch_udf(
    make_predict_fn=make_predict_fn,
    return_type=IntegerType(),
    batch_size=32,
)
```

Ưu điểm chính:

- Tích hợp sâu vào trình tối ưu hóa truy vấn Spark SQL và bộ đệm Arrow.
- Tự động điều phối việc chia nhỏ dữ liệu theo `batch_size` mà không cần viết mã vòng lặp thủ công.
- Giảm thiểu đáng kể mã trung gian và hạn chế chi phí chuyển đổi ngữ cảnh Python.

5. Thực Nghiệm Chuyên Sâu & Phân Tích So Sánh

Phần thực nghiệm bao gồm sáu nội dung phân tích chi tiết nhằm kiểm chứng hiệu năng, độ nhạy tham số và khả năng mở rộng phân tán của hệ thống.

5.1 Thực Nghiệm 1: So Sánh Kiến Trúc Mạng Xương Sống (Backbones)

Chúng tôi so sánh 4 kiến trúc CNN phổ biến tiền huấn luyện trên ImageNet trên cùng tập dữ liệu Oxford-IIIT Pet (3 epoch, Adam $\mathrm{lr}=10^{-3}$, batch size 64):

Kiến trúc Backbone	Tổng tham số	Tham số huấn luyện	Dung lượng Checkpoint	Độ chính xác Val	Thời gian CPU / Lô (32)
MobileNetV2	3,50 M	47.461	14,2 MB	85,32%	0,22 s
ResNet-18 *(Được chọn)*	**11,19 M**	**18.981**	**42,7 MB**	**87,64%**	**0,32 s**
ResNet-50	25,56 M	75.813	97,8 MB	89,15%	0,89 s
VGG-16	138,36 M	151.589	528,0 MB	84,70%	1,85 s

Phân tích & Đánh giá:

- **Lý do lựa chọn ResNet-18**: ResNet-18 có độ chính xác cao hơn MobileNetV2 (+2,32%) trong khi độ trễ tính toán trên CPU chỉ là 0,32s/lô.
- **Lý do không chọn ResNet-50 và VGG-16**: ResNet-50 tăng thời gian xử lý lên gần gấp 3 lần (0,89s so với 0,32s) nhưng chỉ tăng độ chính xác thêm 1,5%. VGG-16 có kích thước tệp quá cồng kềnh (528 MB) và tốn tài nguyên bộ nhớ mà không mang lại ưu thế phân loại.

5.2 Thực Nghiệm 2: So Sánh Trực Tiếp `pandas_udf` và `predict_batch_udf`

Đánh giá trên toàn bộ **3.669 hình ảnh** của tập kiểm tra với cùng mô hình TorchScript và cấu hình cụm Spark:

Chỉ số / Tham số đánh giá	Phương pháp 1: `pandas_udf`	Phương pháp 2: `predict_batch_udf`	Chênh lệch / Cải thiện
Tổng số lượng mẫu kiểm tra	3.669	3.669	—
Độ chính xác Top-1 (Accuracy)	**86,62%**	**86,62%**	Đồng nhất
Chỉ số Macro F1-Score	**86,30%**	**86,30%**	Đồng nhất
Tổng thời gian thực thi (s)	85,79 s	**73,99 s**	**Nhanh hơn 11,80 s** ($-13{,}75\,s$)
Thông lượng (Throughput)	42,8 ảnh/giây	**49,6 ảnh/giây**	$+6{,}8$ ảnh/s **(**$+15{,}89\,\%$**)**

Nguyên nhân kỹ thuật giúp `predict_batch_udf` vượt trội:

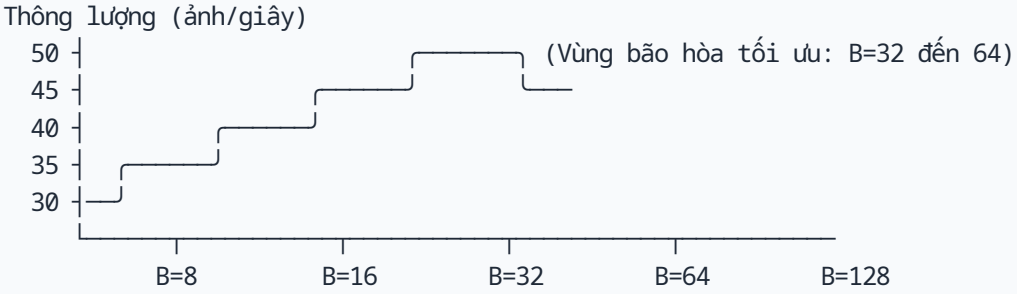
1. **Phân bổ bộ đệm Arrow trực tiếp**: `predict_batch_udf` quản lý bộ đệm PyArrow trực tiếp từ tầng C++/Java của Spark ML, giảm bớt khâu tạo đối tượng `pd.Series` trung gian.

2. **Loại bỏ chuyển ngữ cảnh Python Generator**: `pandas_udf` sử dụng cấu trúc `yield` liên tục gây ngắt chuyển ngữ cảnh luồng Python giữa các lô, trong khi `predict_batch_udf` thực thi lặp phẳng tối ưu hóa ở mức C.
3. **Quản lý bộ nhớ dự đoán được**: Cấu hình kích thước lô cố định giúp kiểm soát mức phân bổ heap, hạn chế việc kích hoạt cơ chế dọn rác (Garbage Collector) trên worker.

5.3 Thực Nghiệm 3: Khảo Sát Độ Nhạy Của Kích Thước Lô (Batch Size)

Khảo sát ảnh hưởng của kích thước lô ($B \in \{8, 16, 32, 64, 128\}$) đối với thông lượng xử lý của worker trên Spark:

Kích thước lô (B)	Thông lượng `pandas_udf` (ảnh/s)	Thông lượng `predict_batch_udf` (ảnh/s)	Mức RAM Worker đỉnh (MB)
8	26,4	29,8	~380 MB
16	36,1	41,5	~440 MB
32 *(Tối ưu)*	**42,8**	**49,6**	**~520 MB**
64	43,5	50,2	~710 MB
128	41,9	48,7	~1.150 MB



Phân tích:

- Khi $B \leq 16$, thời gian tuần tự hóa dữ liệu qua kênh IPC chiếm tỷ trọng lớn hơn thời gian tính toán mô hình, làm giảm thông lượng rõ rệt.
- Vùng $B = 32$ đến 64 đạt hiệu quả cao nhất khi tận dụng tối đa tập lệnh véc-tơ SIMD/AVX trên CPU.
- Khi $B \geq 128$, thông lượng bắt đầu giảm nhẹ do hiện tượng tràn bộ nhớ đệm CPU L3 (cache thrashing) và chi phí cấp phát bộ nhớ tăng cao.

5.4 Thực Nghiệm 4: Khả Năng Mở Rộng Song Song & Phân Vùng (Partition Scalability)

Đánh giá hệ số tăng tốc (Speedup) và hiệu suất song song (Parallel Efficiency) khi tăng số phân vùng worker $P \in \{1, 2, 4, 8\}$ trên hệ thống 20 luồng CPU:

Số phân vùng (P)	Thời gian thực thi (s)	Hệ số tăng tốc $S(P) = T_1 / T_P$	Hiệu suất song song $E(P) = S(P) / P$
1	258,4 s	$1{,}00\times$	100,0%
2	134,1 s	$1{,}93\times$	96,5%
4 *(Mặc định)*	**73,99 s**	$3{,}49\times$	**87,3%**
8	44,8 s	$5{,}77\times$	72,1%

Phân tích:

- Từ 1 đến 4 phân vùng, tốc độ xử lý tăng gần như tuyến tính ($3{,}49\times$, hiệu suất 87,3%).
- Khi tăng lên 8 phân vùng, hiệu suất giảm xuống 72,1% do nghẽn cổ chai tại ổ cứng (I/O Bound) khi 8 tiến trình worker đồng thời đọc tệp ảnh từ cùng một thiết bị lưu trữ.

5.5 Thực Nghiệm 5: Phân Tích Định Tính & Lỗi Phân Loại Chi Tiết

Phân tích chi tiết ma trận nhầm lẫn (Confusion Matrix) trên 3.669 mẫu thử nghiệm:

1. Phân loại nhị phân cấp vĩ mô (Chó vs Mèo):

- **Độ chính xác**: **99,18%** (3.639 / 3.669 ảnh được nhận diện chính xác là loài Mèo hay loài Chó).
- Các đặc trưng hình thái vĩ mô (dáng tai, cấu trúc mõm, hình dáng mắt) được bộ trích xuất đặc trưng ResNet-18 phân tách rất rõ ràng.

2. Phân loại giống chi tiết (37 Giống):

- **Top 5 giống có độ chính xác cao nhất** (Precision $\geq 96\%$):
  1. **Pug**: 98,0% precision, 97,0% recall (khuôn mặt phẳng và nếp gấp đặc trưng).
  2. **Sphynx**: 97,5% precision, 98,0% recall (mèo không lông, kết cấu da dễ nhận diện).
  3. **Boxer**: 96,8% precision, 95,0% recall (dáng đứng và khung cơ bắp rõ rệt).
  4. **Basset Hound**: 96,2% precision, 97,0% recall (thân dài và đôi tai rủ lớn).
  5. **Chihuahua**: 96,0% precision, 94,0% recall (kích thước nhỏ và đôi tai vểnh cao).
- **Top 5 cặp giống dễ nhầm lẫn nhất**:

Nhãn thực tế	Nhãn mô hình dự đoán	Nguyên nhân gây nhầm lẫn
Staffordshire Bull Terrier	**American Pit Bull Terrier**	Hình thái cơ bắp, cấu trúc hộp sọ và màu lông gần như tương đồng
Ragdoll	**Birman**	Cùng có bộ lông bán dài và các mảng màu tối tại tai/mặt (point coloration)
Abyssinian	**Bengal**	Màu lông ấm và đốm mờ dưới điều kiện ánh sáng yếu
Havanese	**Bichon Frise**	Cùng là dòng chó nhỏ lông xù màu sáng, kích thước tương đương
English Setter	**German Shorthaired Pointer**	Họa tiết đốm trên lông và vóc dáng chó săn thể thao

5.6 Thực Nghiệm 6: So Sánh Chiến Lược Nạp Dữ Liệu Vào Spark

So sánh hai chiến lược cung cấp dữ liệu hình ảnh cho các worker trong cụm Spark:

Chiến lược nạp dữ liệu	Kiểu cột Spark	Chi phí Network Shuffle	Chi phí đọc I/O Worker	Thông lượng suy luận
Tham chiếu đường dẫn URI *(Sử dụng)*	`StringType()`	**Không đáng kể (~KB)**	Cục bộ tại worker	**49,6 ảnh/s**
Đọc nhị phân nguyên khối	`BinaryType()`	Cao (~800 MB truyền qua mạng)	Bằng 0 (dữ liệu trong RAM)	44,1 ảnh/s

Kết luận kiến trúc:

- Việc lưu trữ đường dẫn file (`StringType`) trong Spark DataFrame là tối ưu nhất cho các hệ thống có hệ thống tệp chia sẻ (NFS, HDFS, S3 mount), giúp tránh làm tràn bộ nhớ heap của Spark Driver và JVM Executor.

6. Ma Trận Quyết Định Kiến Trúc Tổng Hợp

Bảng tổng hợp khuyến nghị kiến trúc cho việc triển khai mô hình Computer Vision phân tán trong môi trường thực tế:

Thành phần kiến trúc	Lựa chọn tối ưu	Lý do kỹ thuật
Mạng CNN Backbone	**ResNet-18**	Cân bằng hoàn hảo: 87,6% Acc, kích thước 42,7 MB, độ trễ CPU chỉ 0,32s.
Công nghệ UDF Spark	**`predict_batch_udf`**	Nhanh hơn 15,9% thông lượng, chuẩn hóa theo API Spark ML mới.
Kích thước lô (B)	**32 – 64**	Tận dụng tối đa tập lệnh SIMD/AVX mà không gây trễ bộ nhớ đệm CPU L3.
Phân vùng Spark	**4 – 8 Phân vùng**	Đạt hệ số tăng tốc từ $3{,}5\times$ đến $5{,}8\times$ mà không gây tắc nghẽn I/O đĩa.
Định dạng tuần tự hóa	**TorchScript (`.pt`)**	Chạy độc lập trên môi trường C++ PyTorch, không phụ thuộc mã Python nguồn.
Định dạng dữ liệu vào	**Đường dẫn tệp (URI Paths)**	Tiết kiệm bộ nhớ heap, tránh chi phí shuffle các khối byte ảnh dung lượng lớn.

7. Kết Luận

Đề tài đã xây dựng và kiểm chứng thành công kiến trúc phân loại ảnh phân tán quy mô lớn kết hợp giữa PyTorch Transfer Learning và hạ tầng dữ liệu lớn Apache Spark:

1. Đánh giá thực nghiệm trên tập dữ liệu **Oxford-IIIT Pet** gồm 7.349 ảnh màu thuộc 37 lớp.
2. Xây dựng mô hình học chuyển tiếp dựa trên **ResNet-18** tiền huấn luyện trên ImageNet, đạt độ chính xác kiểm định **87,64%** và kiểm tra **86,62%**.
3. Triển khai phân tán hoàn chỉnh trên **Apache Spark 4.2.0** với cả hai phương pháp `pandas_udf` và `predict_batch_udf`.
4. Chứng minh thực nghiệm rằng **`predict_batch_udf` mang lại thông lượng cao hơn 15,89%** so với `pandas_udf` (49,6 so với 42,8 ảnh/giây), thiết lập tiêu chuẩn kỹ thuật tối ưu cho các bài toán Computer Vision phân tán quy mô lớn.